UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Built Like a System.

Gym Membership Management System

E2 – Software Modeling and Design Specifications

Academic Year 2025-2026

1. UML class Diagram -> "What is the system made of?"

The **UML Class Diagram** below presents the main classes of the **Gym Membership Management System**, including their **attributes, methods,** and **relationships** (associations, aggregations).

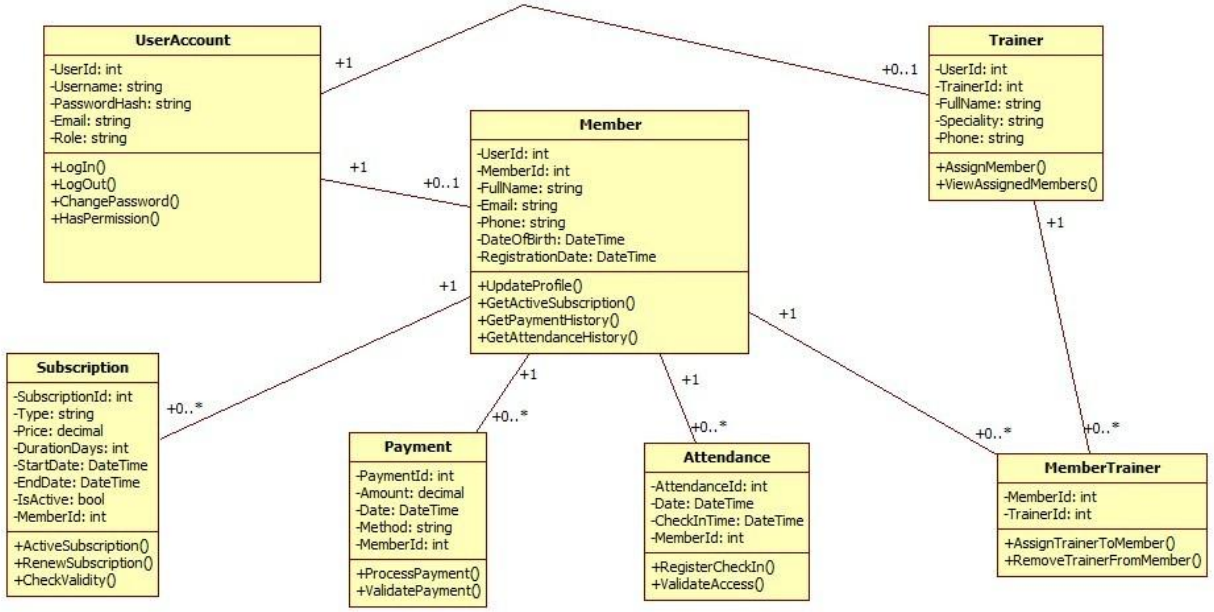

Figure 1 – UML Class Diagram

We added a **UserAccount class** to handle login and roles, like **Admin, Receptionist, Trainer, and Member.** Depending on the role, a user can be linked to a Member or a Trainer, but not necessarily to both. The main functionality is around Members, Trainers, Subscriptions, Payments, and Attendance. A member can have multiple subscriptions, payments, and attendance records, which is why we use **one-to-many relationships.** The connection between Members and Trainers is **many-to-many**, so we used an extra class called MemberTrainer to manage that. We didn't use inheritance because each class has a different role in the system and they don't really extend each other.

Key relationships:

- **UserAccount – Member / Trainer:** One-to-one: each member or trainer has exactly one user account for authentication.
- **Member – Subscription:** One-to-many: a member can have multiple subscriptions over time.
- **Member – Payment:** One-to-many: a member can make multiple payments.
- **Member – Attendance:** One-to-many: a member can have multiple attendance records.
- **Member – MemberTrainer:** Many-to-many through MemberTrainer: a member can be assigned to multiple trainers.
- **Trainer – MemberTrainer:** Many-to-many through MemberTrainer: a trainer can be assigned to multiple members.

2.0 Application Architecture Description -> "How is the system built?"

The system follows **a 3-tier layered architecture** separating concerns into **Presentation, Business Logic, and Data Access layers**, ensuring modularity, maintainability and separation of responsabilities.

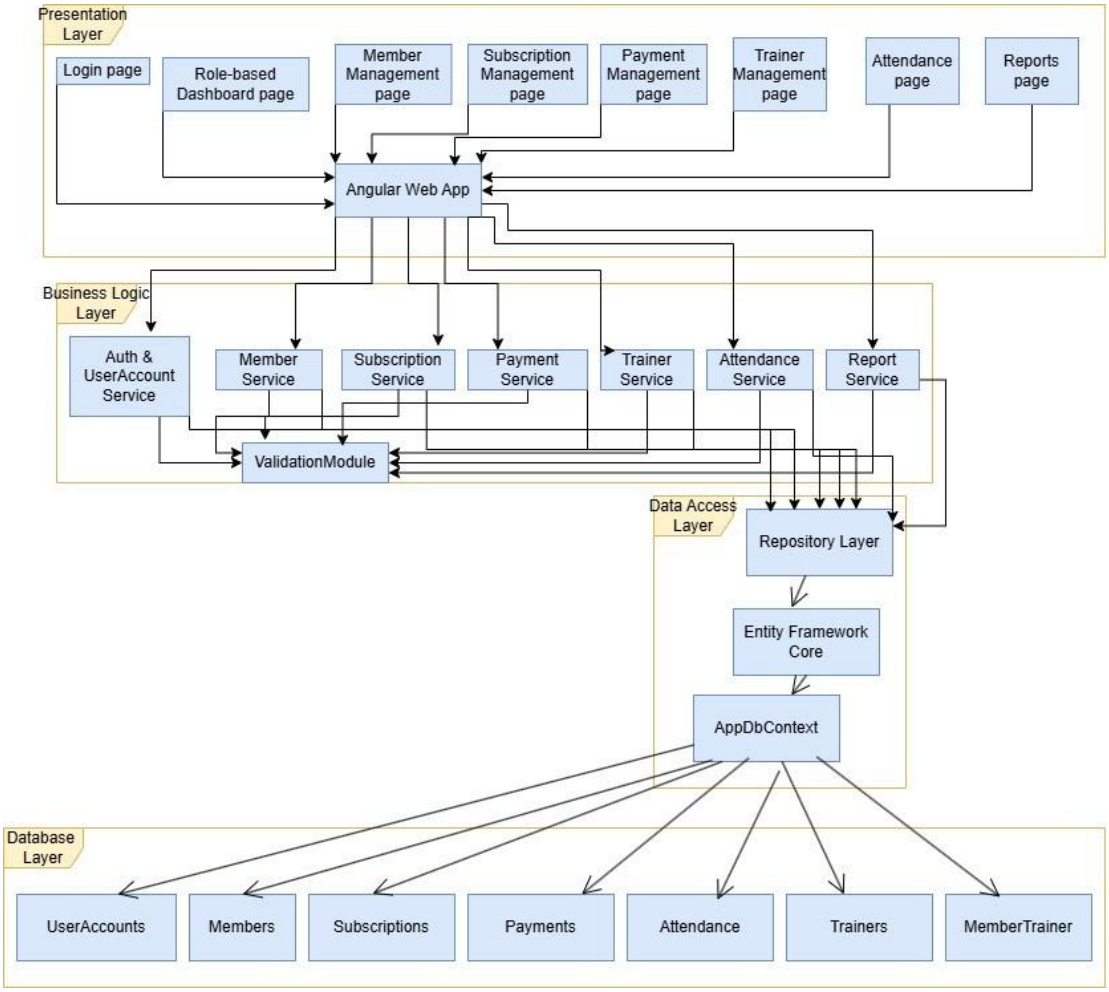

Figure 2 – Architecture Diagram

Layer descriptions:

- **Presentation Layer (Angular Web App):** Contains all UI pages: Login, Role-based Dashboard, Member Management, Subscription Management, Payment Management, Trainer Management, Attendance, and Reports.
- **Business Logic Layer (.NET Services):** Contains the core services: Auth & UserAccount Service, Member Service, Subscription Service, Payment Service, Trainer Service, Attendance Service, and Report Service. All services interact with a shared ValidationModule to ensure data integrity.
- **Data Access Layer (Repository + EF Core):** Implemented using the Repository Pattern on top of Entity Framework Core. The AppDbContext manages the connection and entity mapping to the SQL Server database.

- **Database Layer (SQL):** SQL Server database with tables: UserAccounts, Members, Subscriptions, Payments, Attendance, Trainers, MemberTrainer.

2.1 Entity-Relationship Diagram (ERD) -> "How is the data stored?"

The ERD below shows the database structure with all **entities, primary keys, foreign keys,** and **relationships** between tables.

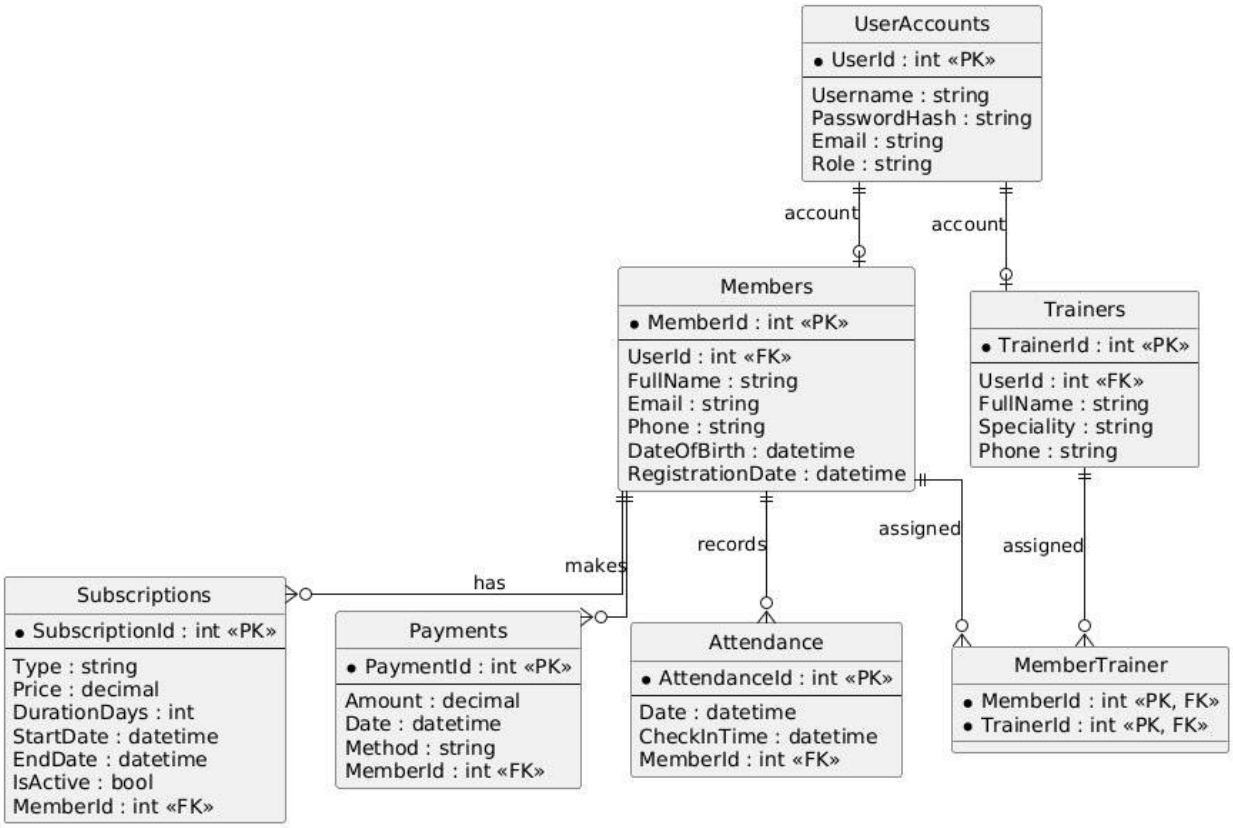

Figure 3 – Entity-Relationship Diagram (ERD)

2.2 Use Case Diagram -> "What does the system do?"

The **Use Case Diagram** illustrates the **interactions between the system actors** (Admin, Receptionist, Trainer, Member) **and** the available system **functionalities.**

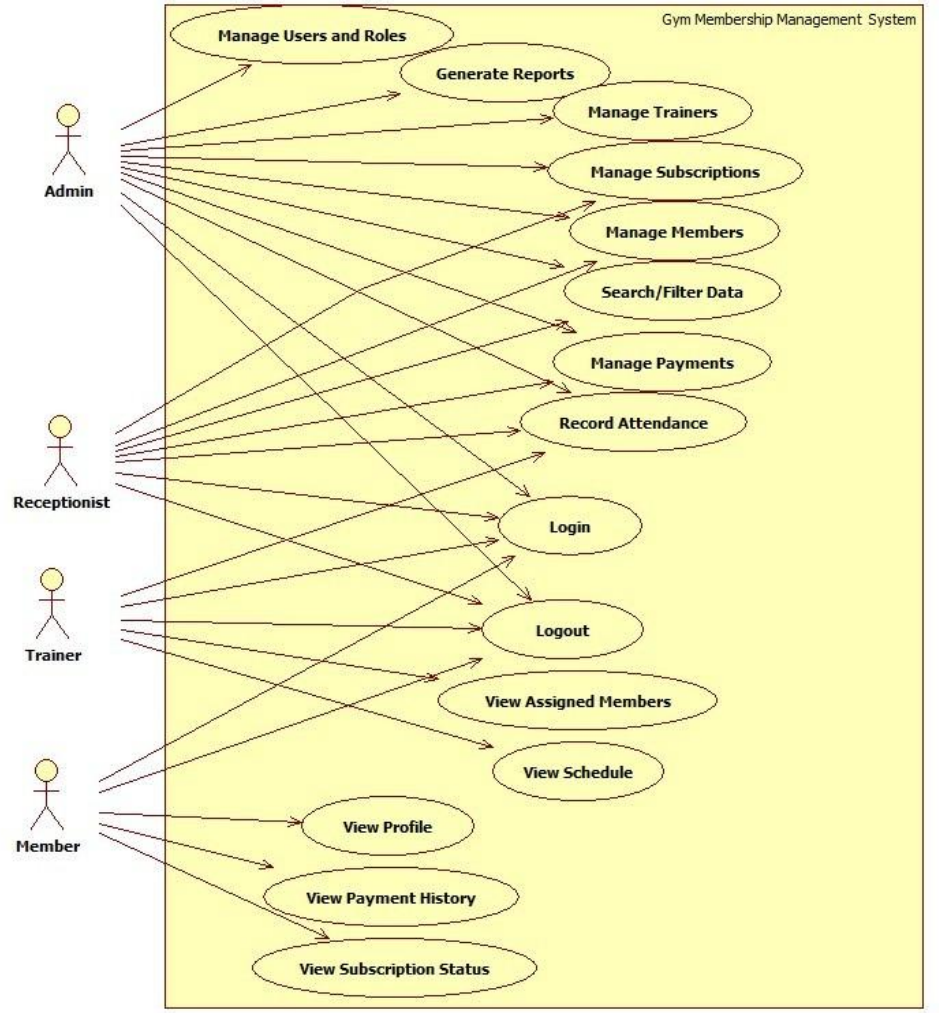

Figure 4 – Use Case Diagram

Actor descriptions:

- **Admin:** Manage members, trainers, subscriptions, payments, attendance, reports, and users/roles.
- **Receptionist:** Add and update members, view subscriptions, register payments, and check attendance.
- **Trainer:** View assigned members, view schedule, and record attendance/sessions.
- **Member:** View personal profile, subscription status, and payment history.

3. Graphical User Interface Design

The **user interface** is designed using **Angular with Figma mockups** as the design reference. The application includes the following main windows/pages:

[PICTURES]

- **Login Page** - authentication form with username and password fields.
- **Sign up Page** - …
- Role-based Dashboard - overview of key metrics, accessible based on user role.
- Member Management Page - list, add, edit, delete, and search members.
- Subscription Management Page - manage subscription plans and member subscriptions.
- Payment Management Page - track and process payments.
- Trainer Management Page - manage trainers and their assigned members.
- Attendance Page - record and view gym check-ins.
- Reports Page - generate and filter reports by date range, status, or plan type.

…just some ideas – depends on which pages are you done with till MAY 8th

[Login Page and App Shell mockups designed in Figma by Salajanu Mircea Vasile – Frontend Developer]

4. Description of the Main Classes

The following table describes **each main class**, its role in the system, and its key responsibilities:

Class	Role	Responsibilities
UserAccount	Authentication & Access Control	Manages user credentials and roles (Admin, Receptionist, Trainer, Member). Handles login, logout, password management, and access permissions.
Member	Core Domain Entity	Represents a gym member. Stores personal details and links to subscriptions, payments, attendance records, and assigned trainers.
Subscription	Membership Management	Tracks the type, price, duration, and validity of a member's subscription. Ensures only active subscriptions are used.
Payment	Financial Transactions	Records payment transactions made by members. Processes and validates payments linked to subscriptions.
Attendance	Check-in Tracking	Records each gym visit by a member. Validates access based on active subscription status.
Trainer	Trainer Management	Stores trainer details and specialization. Allows viewing and managing assigned members.
MemberTrainer	Many-to-Many Relationship	Junction entity linking members to their assigned trainers. Manages assignment and removal of trainers for members.

5. Test Plan

The test plan defines the **key test cases for the system**. Each test case specifies the feature being tested, the **input values** used, and the **expected result**. Tests are defined by the QA Engineer (Luidort Zsuzsa).

just examples complete as you think

TC#	Feature	Input Values	Expected Result
TC01	Login – valid credentials	Username: admin Password: Admin123!	Authenticated; redirected to Dashboard.
TC02	Login – invalid password	Username: admin Password: wrongpass	Error message; user not authenticated.
TC03	Login – empty fields	Username: (empty) Password: (empty)	Validation error; login not attempted.
TC04	Add Member – valid data	FullName: Ion Popescu Email: ion@test.com Phone: 0740123456	Member added and visible in the list.
TC05	Add Member – duplicate e	nailEmail already in DB	Error: 'Email already in use'; not saved.
TC06	Add Member – missing fiel	dsFullName: (empty)	Validation error; member not saved.
TC07	Subscription – add valid pl	anType: Monthly Price: 50 Duration: 30 days	Subscription plan saved in the list.
TC08	Subscription – invalid price	Price: -10	Validation error: price must be positive.
TC09	Payment – process valid p	aymentAmount: 50 Method: Cash MemberId: 1	Payment recorded; subscription updated.
TC10	Attendance – valid check-ir	MemberId: 1 (active subscription)	Check-in recorded with correct timestamp.
TC11	Attendance – expired subs	criptionMemberId with expired subscription	Access denied; check-in not recorded.
TC12	Trainer – assign to membe	rTrainerId: 2 MemberId: 1	Trainer-member link created.
TC13	Reports – filter by date ran	geStart: 01/01/2026 End: 31/01/2026	Only records within range displayed.
TC14	Role access – Receptionist	Login as Receptionist	Access to Members, Attendance, Payments only.
TC15	Logout	User clicks Logout	Session cleared; redirected to Login page.

Built Like a System.